

LLM Prompts Outside the Translation Benchmark

This document records the model instructions used by the experiment pipeline outside `prompts/benchmark/`. The benchmark translation prompts and their paper origins remain documented in [prompts/benchmark/README.md](#) and in the [benchmark methodology](#).

The templates below are the core text sent by OxCidation. Values in braces are replaced at runtime. For structured calls, the provider also receives the output schema listed after the prompt. If strict structured output is unavailable, LangChain prepends schema-derived JSON formatting instructions.

Prompt Inventory

Operation	Responsible agent	When it runs	Implementation
Default initial translation	Translator	Main pipeline runs without an explicit benchmark prompt	<code>prompts/direct_translation_prompt.txt</code>
Visible-test generation	Tester	Once per selected C program, before translation prompts	<code>prompts/visible_test_generation_prompt.txt</code>
Visible-test replacement	Tester, using deterministic or Validator requirements	For failed or explicitly replaced slots, within the preparation bound	Appended in <code>src/server.py</code>
Visible-suite correctness review	Validator	After deterministic candidate filtering and before translation	<code>prompts/visible_test_review_prompt.txt</code>
Translation-discrepancy analysis	Validator	After C passes a visible case and Rust fails it	Built in <code>src/server.py</code>
Rust repair	Translator	After compilation failure or a confirmed translation discrepancy	Built in <code>src/server.py</code>

1. Default Initial Translation

The benchmark normally supplies one template from `prompts/benchmark/`. This fallback is used only when no explicit translation prompt is supplied. Its text is identical to the benchmark control `direct-v1`.

```
Your task is only to translate C code to Rust code.
It should be a direct translation of the C code, so before producing the final Rust
code, verify internally:
1. Same input format?
2. Same output format?
3. Same branches?
4. Same loop bounds?
5. Same arithmetic formulas?
6. Same indexing behavior?
7. Same algorithm and data representation?
8. No optimizations or redesigns?
9. Same logging?

C Code:
{c_code}
```

The structured response contains `rust_code`. On the JSON fallback path, the system additionally prepends `You are an expert C to Rust translator.` and the schema-generated formatting instructions.

2. Visible-Test Generation

The Tester receives only the accepted C source. It does not receive the problem statement, Rust translation, sample I/O, or Judge cases.

```
You are the Tester Agent for a behavior-preserving C-to-Rust translation pipeline.

Using only the C source code below, generate a language-agnostic batch of concrete
standard-input cases that can help evaluate the program's observable behavior.

Choose the number of cases needed for this program. Cover distinct meaningful
behaviors that can be inferred from the source. Avoid redundant cases, give every
case a clear purpose, and make each input complete for the execution it is intended
to exercise.

Do not assume a problem statement or constraints that are not present in the source.
Return only the structured test batch. Do not generate test code, commands, expected
outputs, or malformed inputs. Expected outputs are obtained separately by executing
the C reference program.

C source:
{c_code}
```

Structured response:

```
strategy: string
cases:
  - purpose: string
    input: string
```

3. Visible-Test Regeneration

If slots fail deterministic validation or the Validator requests replacements, the following text is appended. Retained cases, replacement requirements, and forbidden prior inputs are provided as structured JSON.

```
Generate only the requested replacement cases, one per slot and in the listed order.
Do not repeat retained cases or reuse rejected inputs.
Replacement request: {regeneration_feedback}
```

The JSON request contains `replacement_slots`, `retained_cases`, and `forbidden_inputs`. Deterministic failures use generic requirements and do not expose runtime diagnostics to the Tester. Validator revisions use only the actionable replacement requirements returned for the named cases. The orchestrator maps the ordered responses back to the stable slot identifiers.

4. Visible-Suite Correctness Review

The Validator reviews the complete executable batch, not each case in isolation. It sees the C source, generated inputs, and a compact deterministic summary. It does not see Rust, expected outputs, or Judge evidence.

You are the Code Validator in a behavior-preserving C-to-Rust translation pipeline.

Review the complete batch of generated inputs using only the C source code and the deterministic execution summary.

Decide whether the batch is a useful and coherent set of inputs for evaluating the program. Recommend replacements only for concrete, source-supported problems that materially reduce the batch's usefulness, such as an invalid input, substantial redundancy, or a clearly missing distinct behavior.

Preserve as many existing cases as possible. Do not require exhaustive coverage, assume a problem statement, or invent input constraints that cannot be inferred from the source.

If revision is needed, identify only the cases that should be replaced and provide concise requirements for each replacement. Do not generate replacement inputs.

C source:
{c_code}

Generated input batch:
{cases}

```
Deterministic C execution summary:
{deterministic_report}
```

Structured response:

```
assessment: approved | revise
replacements:
  - case_id: string
    reason: string
    requirements: string
```

An approved batch must have no replacements. A revision must name existing case identifiers and provide both a reason and actionable requirements. An LLM or parsing failure is recorded separately as `review_failed`.

5. Translation-Discrepancy Analysis

This prompt runs only after a differential visible failure where C passed and Rust failed. It verifies the evidence before any semantic repair is allowed.

```
You are a code validation specialist for behavior-preserving C to Rust translation.
Analyze the C source, Rust translation, and differential visible-test report. First
verify that the failing inputs are complete and valid for every input operation
performed by the C source. If an input is malformed, incomplete, or makes the C
oracle depend on undefined or uninitialized data, classify it as
invalid_visible_test and do not recommend changing Rust to accept it. If the
available evidence cannot support either conclusion, classify it as inconclusive.
Otherwise classify it as translation_discrepancy and provide concise behavior-
preserving repair guidance. Identify only supported discrepancies. Do not reveal
hidden chain-of-thought.
```

```
C source:
{c_code}
```

```
Rust translation:
{rust_code}
```

```
Differential test report:
{test_report_as_json}
```

Structured response:

```
test_assessment: translation_discrepancy | invalid_visible_test | inconclusive
diagnosis: string
repair_guidance: string
semantic_discrepancies: list[string]
```

Only `translation_discrepancy` can send the pipeline to Rust repair.

6. Rust Repair

Compilation diagnostics or confirmed semantic repair guidance are inserted into the same repair template.

```
You are an expert C-to-Rust translator. Repair the Rust translation while preserving
the C program's behavior and intended algorithm.
Failure category: {failure_category}

C source:
{c_code}

Current Rust translation:
{rust_code}

Issues:
{errors}

Return ONLY fixed Rust code without markdown.
```

The model response contains repaired Rust code. The MCP transport also carries token usage metadata for reporting. The configured repair limit determines whether the pipeline may call this prompt again; reaching it proceeds to final Judge observation without another repair.

Inactive Prompt File

`prompts/translation_prompt.txt` is retained in the repository but has no runtime reference in the current implementation. It must not be described as an active experiment prompt unless the code is changed to use it.